

Transformer 之所以能取代 RNN/LSTM，最大的原因就是它抛弃了“时间步 (Time Step)”的循环依赖，实现了训练时的并行化。

1. 痛点：为什么 RNN 无法并行？

在 RNN（或 LSTM）中，网络的计算是串行的。

假设我们要训练一句话：“我 爱 机器人”。

- **t=1:** 输入“我”，计算隐藏层 h_1 ，预测“爱”。
- **t=2:** 输入“爱”，必须拿到 h_1 ，才能计算 h_2 ，预测“机器人”。
- **t=3:** ...

瓶颈：你如果不把第 1 步算完，第 2 步就根本没法开始。无论你有多个 GPU，你也只能等着前一步算完。这就像接力跑，必须等上一棒交接。

2. Transformer 的并行训练

Transformer 的结构里没有“循环”。它的输入不是一个一个进来的，而是整个句子的矩阵 (Matrix) 一次性“拍”进网络里的。

核心机制 A：Encoder 的并行（天然并行）

Encoder 负责“理解”句子。

- **输入：**形状为 (Batch_Size, Sequence_Length, Embedding_Dim) 的大矩阵。
- **处理：**Self-Attention 机制是通过矩阵乘法实现的。
 - 比如计算 Attention Score，是 $Q \times K^T$ 。
 - 这意味这，单词“我”去注意“机器人”，和单词“机器人”去注意“我”，这两个计算过程是同时发生在矩阵运算里的。
 - GPU 最擅长做这种大规模矩阵乘法。

结论：Encoder 在处理输入的 100 个单词时，是 100 个单词同时处理，不需要排队。

核心机制 B：Decoder 的并行（Teacher Forcing + Mask）

这是最难理解的部分。Decoder 负责生成句子。

你可能会问：“生成句子不是必须一个一个崩出来吗？先生成‘爱’，才能生成‘机器人’啊？”

推理时 (Inference)：是的，必须串行。

训练时 (Training)：不是！我们可以并行。

这里用到了 **Teacher Forcing** 技术。

场景：

- **输入 (Input)：** <sos> 我 爱 机器人
- **目标 (Target)：** 我 爱 机器人 <eos>

在训练时，我们已经知道正确答案了（上帝视角）。我们不需要等模型自己瞎猜出“爱”字再喂给下一步。我们可以直接把正确的标准答案 (Ground Truth) 喂给 Decoder。

训练过程如下（一步到位）：

- 1. **输入准备：** 我们把整个句子 `<sos> 我 爱 机器人` 作为输入矩阵 X 一次性喂给 Decoder。
- 2. **并行预测：** 模型在**同一时刻**输出了 4 个预测结果。
 - 位置 1 的输出：预测 `<sos>` 后面是啥？（目标：我）
 - 位置 2 的输出：预测 `我` 后面是啥？（目标：爱）
 - 位置 3 的输出：预测 `爱` 后面是啥？（目标：机器人）
 - 位置 4 的输出：预测 `机器人` 后面是啥？（目标：`<eos>`）
- 3. **计算 Loss：** 这 4 个预测位置的 Loss 是同时计算出来的，然后取平均。

3. 既然并行输入了，怎么防止“作弊”？（Causal Masking）

如果把 `<sos> 我 爱 机器人` 一次性全喂进去，预测“我”后面的字时，模型利用 Self-Attention 可能会偷看到后面的“爱”字。这在训练时属于作弊（Leakage），会导致测试时（没有后续字）模型失效。

为了解决这个问题，Transformer 引入了 **Masked Self-Attention（掩码注意力）**。

这是一个**上三角矩阵**（Upper Triangular Mask），里面的值为 $-\infty$ （负无穷）。

位置	可以看到 T1	可以看到 T2	可以看到 T3	可以看到 T4
预测 T1 (我)	✓	⊘ Mask	⊘ Mask	⊘ Mask
预测 T2 (爱)	✓	✓	⊘ Mask	⊘ Mask
预测 T3 (机器)	✓	✓	✓	⊘ Mask
预测 T4 (EOS)	✓	✓	✓	✓

在 Softmax 之前加上这个 Mask：

- 当模型在位置 1 计算 Attention 时，位置 2, 3, 4 的分数会被加上 $-\infty$ ，Softmax 之后概率变为 0。
- **物理意义：** 虽然整个句子都在矩阵里，但我强行把这一行后面的数据“抹黑”，让模型看不见未来。

4. 总结：并行训练流程图

- 1. **数据准备：** 拿到一整个 Batch 的句子。
- 2. **Embedding：** 把所有单词同时转成向量。
- 3. **Add Positional Encoding：** 把位置信息加进去（因为没有时间步了，得手动加位置标记）。
- 4. **Encoder：** 全并行运算，通过矩阵乘法提取特征。
- 5. **Decoder 输入：** 放入整个目标句子（Shifted Right，即右移一位）。
- 6. **Masking：** 盖住未来，防止作弊。
- 7. **Decoder 输出：** 一次性算出所有位置的预测概率分布。
- 8. **Loss：** 一次性拿所有位置的预测结果和真实标签做 CrossEntropy，算出总 Loss。

这就是为什么 Transformer 训练极其快，能吃满 GPU 显存，而 RNN 只能在那慢吞吞地磨洋工。

这是一个非常棒的问题！搞清楚这点，你对 Transformer 的理解就超越了 80% 的初学者。

推理时： 是“摸着石头过河”，上一步生成的字是下一步的输入，**串行**计算（Auto-regressive）。

2. 推理模式 (Inference Mode)

核心策略：Auto-regressive（自回归）

在推理（应用）时，我们没有答案。模型必须依赖自己上一步的输出来决定这一步走哪里。这是一个**死循环**，直到模型生成结束符 `<EOS>`。

- **Step 1:**
 - **输入：** 只有 `<SOS>`。
 - **处理：** Decoder 运算，输出概率最高的词 -> “我”。
- **Step 2:**
 - **输入：** 把上一步生成的“我”拼接输入序列中 -> `<SOS>` 我。
 - **处理：** Decoder 重新运算整个序列（或者利用 KV Cache 优化），输出下一个词 -> “爱”。
- **Step 3:**
 - **输入：** 再次拼接 -> `<SOS>` 我 爱。
 - **处理：** Decoder 运算，输出 -> “机器人”。
- **Step 4:**
 - **输入：** `<SOS>` 我 爱 机器人。
 - **处理：** Decoder 运算，输出 -> `<EOS>`。
- **终止：** 遇到 `<EOS>` 停止。

推理时，在预测第k个token时，也是基于当前1~k位的输入，一次性预测1~k位的输出，其中1~k-1位输出是基于1~k-1位输入做出的，和之前的重复，我们只取最后一位输出加到生成的句子末尾。

训练时和推理时，输入和输出的矩阵形状（Shape）是不一样的。

但是，**模型的参数（权重矩阵）是不变的**。Transformer 之所以能处理不同形状的输入输出，是因为它在设计上对“序列长度（Sequence Length）”这个维度是“弹性”的。

1. 维度对比：训练 vs 推理

假设：

- **B (Batch Size)** = 1（为了简单，假设只有一句话）
- **L (Sequence Length)** = 句子长度
- **V (Vocab Size)** = 词表大小（比如 10000 个词）
- **D (Hidden Dim)** = 隐藏层维度（比如 512）

A. 训练时 (Training)

训练是**并行**的，输入是固定长度（通常是经过 Padding 补齐的最大长度，比如 10）。

- **输入形状:** $(1, 10, D)$ —— 包含 `<sos>` 我 爱 机 器 人 `<pad>...`
- **模型处理:** 每一层都保持这个长度 10。
- **输出形状:** $(1, 10, V)$
- **含义:** 模型同时吐出了 10 个位置的预测结果。
 - 第 1 个位置预测 `<sos>` 的下一个词。
 - 第 2 个位置预测 我 的下一个词。
 - ...
- **结果:** 我们计算 Loss 时，会用到这整个 $(1, 10, V)$ 的大矩阵。

B. 推理时 (Inference)

推理是**串行**的，序列长度 L 是**动态增长**的。

- **第 1 步:**
 - 输入: `<sos>`
 - 输入形状: $(1, 1, D)$
 - 输出形状: $(1, 1, V)$ → 取最后一个词，得到“我”
- **第 2 步 (Naive 模式):**
 - 输入: `<sos>` 我
 - 输入形状: $(1, 2, D)$
 - 输出形状: $(1, 2, V)$
 - **关键点:** 这里输出了 2 个结果！
 1. `<sos>` 下一位的预测（是“我”，这个我们早就知道了，**废弃**）。
 2. 我 下一位的预测（是“爱”，这是我们要的，**保留**）。
 - 我们只取矩阵的**最后一行**（Last Token）。
- **第 3 步:**
 - 输入: `<sos>` 我 爱
 - 输入形状: $(1, 3, D)$
 - 输出形状: $(1, 3, V)$ → 废弃前两个，只取最后一个。

2. 为什么模型不会报错？（全连接层的特性）

你可能会担心：“最后那个全连接层（Linear Layer），不是需要固定的输入维度吗？”

PyTorch 中的 `nn.Linear(in_features, out_features)` 实际上是非常灵活的。

- 它只关心**最后一个维度**是否匹配。
- 它不管前面有多少个维度（Batch 维度、Time 维度）。

假设你的输出层定义为 `nn.Linear(512, 10000)`。

- **训练时：**输入是 `(1, 10, 512)`。Linear 层会把前两个维度 `(1, 10)` 看作 $1 \times 10 = 10$ 个独立的样本，分别进行矩阵乘法。输出变成 `(1, 10, 10000)`。
- **推理第 3 步：**输入是 `(1, 3, 512)`。Linear 层把它们看作 $1 \times 3 = 3$ 个样本。输出变成 `(1, 3, 10000)`。

结论：只要隐藏层维度 `512` 对得上，序列长度是 1 还是 100，Linear 层都不在乎。这就是 Transformer 能够处理变长输入的数学基础。

“Right Shifted”(右移一位) 是 Transformer **训练数据预处理** 中一个非常关键的步骤，特指 **Decoder 的输入 (Input)** 和 **标签 (Label/Target)** 之间的错位关系。

简单来说：**Decoder 的输入**，就是把正确答案整体向右挪了一格，并在最前面塞进去一个开始符 `<SOS>`。

1. 为什么要右移？（核心逻辑）

我们训练模型的目的是：**让它根据“上一个字”，预测“下一个字”**。

- 当我们想让模型预测第 2 个字时，我们必须把第 1 个字喂给它。
- 当我们想让模型预测第 3 个字时，我们必须把第 2 个字喂给它。

这就形成了一种**错位**的对应关系。

2. 图解实例

假设我们要训练一句话：“我 爱 机器人”。

加上开始符 和结束符，完整的序列是：

`['<SOS>', '我', '爱', '机器人', '<EOS>']`

在训练时，我们把这个序列拆成两半：

时间步	Decoder 输入 (Input)(Right Shifted)	真实标签 (Target/Label)(期望模型输出的)	逻辑含义
1	<code><SOS></code>	我	看到“开始”，预测第一个字是“我”
2	我	爱	看到“我”，预测下一个字是“爱”
3	爱	机器人	看到“爱”，预测下一个字是“机器人”
4	机器人	<code><EOS></code>	看到“机器人”，预测这句话讲完了

仔细看上面的表：

- **输入列：**`['<SOS>', '我', '爱', '机器人']`

- **标签列:** ['我', '爱', '机器人', '<EOS>']

你会发现，**输入列**实际上就是把**标签列**整体**向右移动了一位**，然后在空出来的最左边补了一个 **<SOS>**。

这就是 "**Right Shifted**" 的由来。

3. 代码里是怎么做的？

在 PyTorch 的具体实现中，这通常通过切片（Slicing）操作来实现。

假设 **batch** 里的数据是完整的 **[<SOS>, A, B, C, <EOS>]**：

Python

```
# 假设 full_sentence = [<SOS>, 我, 爱, 机器人, <EOS>]

# 1. Decoder 的输入（去掉最后一个 <EOS>）
decoder_input = full_sentence[:-1]
# 结果: [<SOS>, 我, 爱, 机器人] -> 这就是 Right Shifted

# 2. 真实的标签（去掉第一个 <SOS>）
target_label = full_sentence[1:]
# 结果: [我, 爱, 机器人, <EOS>]
```

4. 总结

“Right Shifted”并不是什么复杂的数学变换，它只是为了构造 **Supervised Learning（监督学习）** 的数据对：

- **Question:** 你看到了什么？（当前序列，以 **<SOS>** 开头）
- **Answer:** 下一位应该是什么？（目标序列，以 **<EOS>** 结尾）

这种错位构造，配合 **Causal Mask**，让 Transformer 能够在一次并行计算中，学会所有位置的“预测下一个词”的任务。